Playable but Private: In-Encoder Selective Encryption with Ascon-AEAD128 and Attack-Adaptive Protection for MJPEG Video on an FPGA SoC

Aravind R, Suhas V J, Rishab H R, Samarth B N, Vijaysurya B S, Chetana G [TODO: order]
Department of Electronics and Communication Engineering, RV College of Engineering, Bengaluru, India
[TODO: e-mails]

***Abstract*—We present a JPEG/MJPEG encoder for a Zynq-7020 FPGA SoC that encrypts the sign and amplitude bits of DCT coefficients while they are entropy-coded, with a keystream from the NIST lightweight standard Ascon-AEAD128. Code lengths do not change, so every protection level is a valid JPEG, and a 128-bit tag over the frame metadata and scan rejects modified, replayed and forged frames. The strongest level hides block positions with a keyed per-frame shuffle of restart segments that needs no frame buffer: the fabric codes and authenticates the segments and the ARM core only reorders them. The design is bit-exact with a golden model in simulation and on a PYNQ-Z2 board in all six levels, adds no cycles for encryption, closes timing at 100 MHz and reaches 176 Mpixel/s with four lanes. On 9 sources, four re-implemented published schemes leak 0.58–0.94 edge correlation under the same attacks, while our shuffle level stays at or below 0.02, also against jigsaw solvers and a trained U-Net. An authenticated feedback loop mitigates all 11 modelled network attacks in 0.91 s on average with no false alarms.**

***Index Terms*—selective encryption, MJPEG, Ascon, FPGA, authenticated encryption, video surveillance**

I. Introduction

Cameras on doors, corridors and machines stream over networks they do not control. The viewer needs confidentiality (no bystander watches the scene) and integrity (no frame is edited, replayed or injected). Encrypting the transport with TLS or SRTP [1] gives both, but hides the stream from every standard tool and costs cipher work on every byte. Selective encryption instead encrypts the parts of the compressed stream that carry most of the picture and keeps the format [2]: for JPEG, the sign and amplitude bits of DCT coefficients can be changed without changing any code length [3], [4]. Such schemes are weaker than they look, because the positions of non-zero coefficients stay public [2], [5].

We ask how much of a secure camera link fits into the encoder hardware, and what an eavesdropper still learns. Our contributions are:

1) An MJPEG encoder in Verilog that XORs an Ascon-AEAD128 [6] keystream onto coefficient fields during entropy coding, with zero added cycles, a per-frame tag over metadata and scan, and a lane variant at 176 Mpixel/s (Section III).
2) A hardware form of a keyed macroblock shuffle (E5): the fabric codes one restart segment per macroblock, the processor reorders them, and the tag binds the order.
3) An evaluation of six protection levels and four published schemes under one benchmark of keyless attacks, including a trained U-Net, on 7 clips and the Kodak set (Section V).
4) An authenticated feedback loop with a forward-secure ratchet that raises protection under attack (Section VI).

II. Related Work

Van Droogenbroeck and Benedett encrypted the appended bits of AC coefficients of baseline JPEG [3]; Auer et al. permuted and toggled bitstream fields at 25 VGA frames/s [4]. Massoudi et al. warn that visual distortion is not security [2]; Lin et al. recover DCT sign bits by linear programming [5]; Hirose et al. show that shuffling blocks delimited by restart markers leaks unless the restart interval is small [7]; Li et al. recover block shuffles with a chosen plaintext [8]. In hardware, Kishore et al. scrambled blocks and symbols inside an FPGA JPEG encoder at 13–17 % more ALUTs [9], and Bağbaba and Örs added TEA encryption of DC and low AC coefficients [10]; neither authenticates frames, and both use fixed key material. Ascon is small in hardware (7 kGE for one unrolled round) [11], [12], and has been built on Artix-7 with replay detection [13]. Frame sizes of encrypted streams identify the video [14]. We found no design that puts a standardized AEAD inside a hardware JPEG encoder, keeps every level playable while authenticating each frame, and closes the loop from receiver to encoder.

III. System Design

Threat model. The attacker on the network path observes all traffic and may modify, drop, delay, replay or inject video and feedback datagrams; it holds no key. Sender and receiver share a master key K_{m} provisioned at installation; key distribution, physical and side-channel attacks are out of scope.

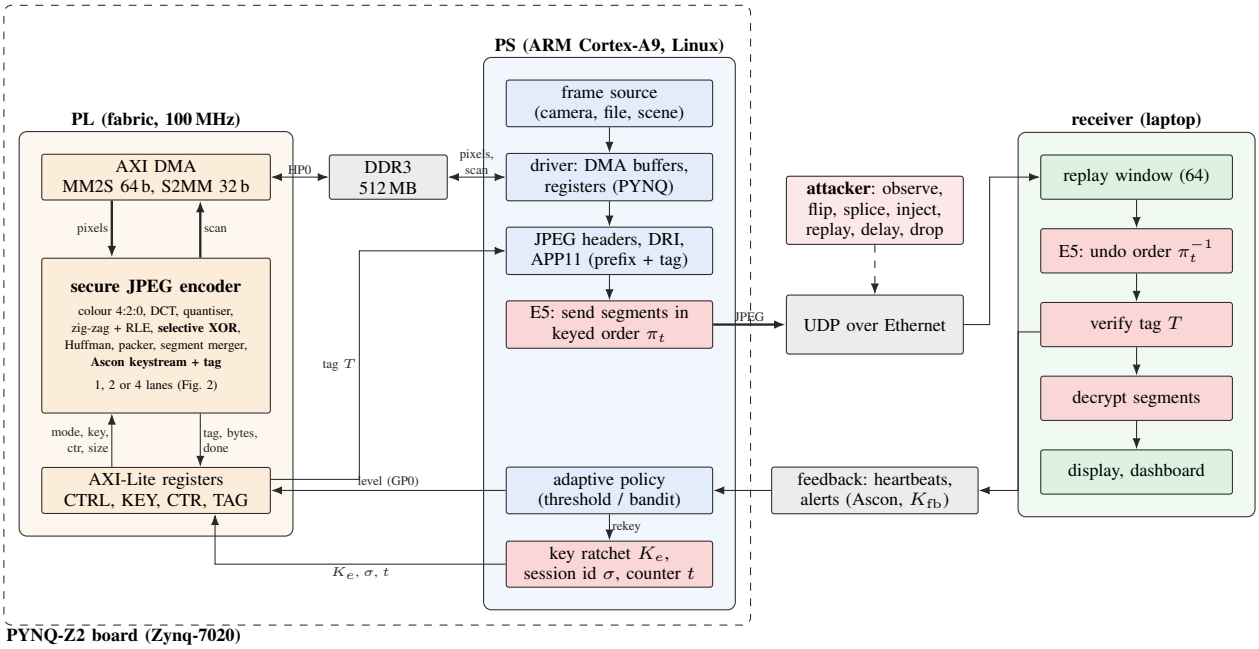


Fig. 1. System architecture. The fabric (PL) encodes, encrypts and tags; the ARM core (PS) adds headers, reorders E5 segments and runs the key ratchet and policy; the receiver undoes the order, verifies the tag and decrypts, and reports back over an authenticated feedback channel. Shaded red: cryptographic operations.

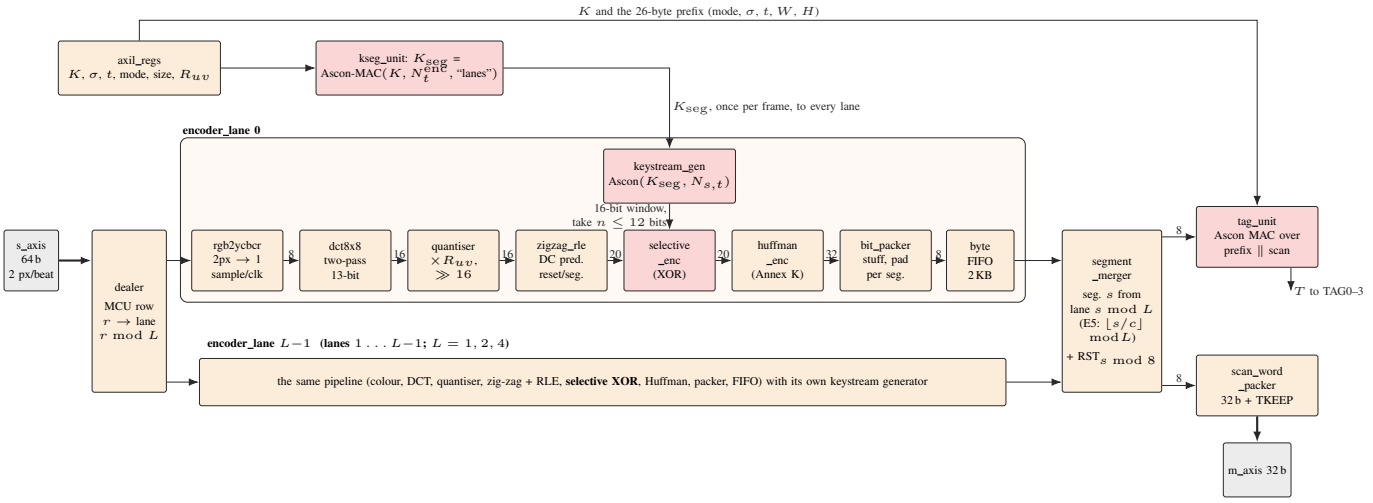


Fig. 2. Fabric datapath of the lane encoder (`secure_jpeg_lanes_top`); numbers on arrows are bus widths in bits. MCU row r goes to lane $r \bmod L$; every lane restarts DC prediction and its keystream per restart segment; the merger restores raster order and inserts restart markers; the tag unit authenticates the merged scan.

Encryption inside the entropy coder. For a quantized coefficient $v \neq 0$ JPEG writes a Huffman code for (run, s), $s = \lfloor \log_2 |v| \rfloor + 1$, followed by the s-bit field

$$\mathcal{A}(v) = \begin{cases} v, & v > 0 \\ v + 2^s - 1, & v < 0 \ . \end{cases} \tag{1}$$

Given keystream bits $z_0 z_1 \ldots$, a selected symbol is encrypted as

$$\mathcal{A}' = \mathcal{A} \oplus z_0(2^s - 1) \oplus \alpha \textstyle\sum_{k=1}^{s-1} z_k 2^{k-1}, \tag{2}$$

where $\alpha = 1$ if amplitudes are selected. Since $\mathcal{A}(v) \oplus (2^s - 1) = \mathcal{A}(-v)$, the first term is exactly a sign change; the second changes only the $s-1$ low bits, so $|v|$ stays in $[2^{s-1}, 2^s - 1]$. Hence s, the Huffman code and the stream length never change, and the output is a valid JPEG for any key. Fig. 3 shows the gate-level circuit; it consumes $n = [\text{sign}] + \alpha(s - 1) \le 11$ bits per symbol in one clock from a 16-bit window, while the Ascon core delivers 128 bits every 9 cycles, so the cipher never stalls the encoder.

Keys, nonces and tag. The video key follows a one-way ratchet, $K_{e+1} = \mathrm{KDF}(K_e, \text{ratchet}, e{+}1)$, with KDF the

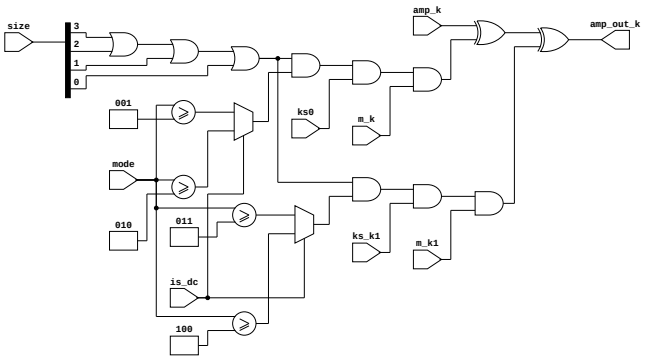


Fig. 3. Gate-level circuit of bit k of the encryption stage, generated from the RTL (Yosys, netlistsvg): the comparators and multiplexers (select: is_dc) form the sign and amplitude enables, and two XORs apply (2). $m_k = [k < s]$; ks0, ks_k1: keystream bits 0 and $k{+}1$. Eleven such slices form the stage.

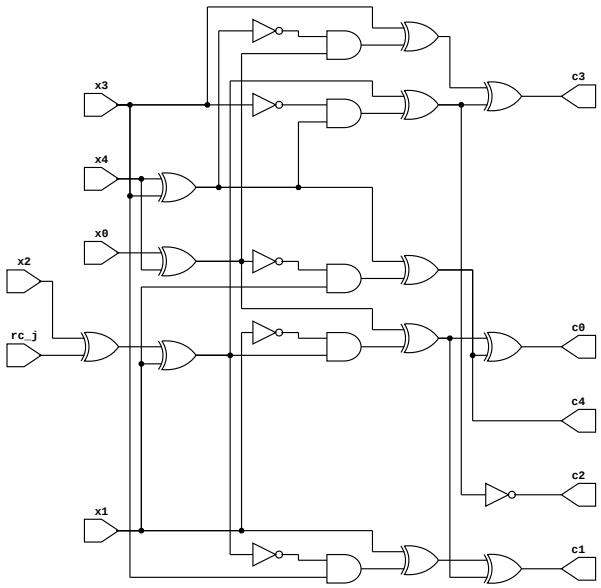


Fig. 4. One of 64 bit slices of the Ascon round in our RTL (p_C and p_S), generated from the Verilog: 12 XOR, 5 AND, 6 NOT gates; one round per clock cycle.

Ascon tag under a reserved nonce; a separate key K_{fb} protects feedback. The 128-bit nonce is $N(\sigma, t, b) = \sigma \,\|\, t \,\|\, b$: a 64-bit session identifier σ (48 random bits per boot and a 16-bit epoch), a 63-bit frame counter t and a purpose bit b. Each frame derives a segment key and authenticates its scan:

$$K_{\mathrm{seg}} = \mathrm{MAC}_{K_e}\big(N(\sigma, t, 0), \text{“lanes”}\big), \tag{3}$$

$$Z_s = \mathcal{E}_{K_{\mathrm{seg}}}\big(N(s, t, 0), 0^{\ell}\big), \tag{4}$$

$$T_t = \mathrm{MAC}_{K_e}\big(N(\sigma, t, 1), P_t \,\|\, \mathrm{scan}_t\big), \tag{5}$$

where Z_s is the keystream of restart segment s and P_t is a 26-byte prefix (version, level, σ, t, width, height). Binding the level stops downgrades; (K, N) never repeats because t increases, a rekey changes K_e and σ, and two boots share σ with probability 2^{-48} per pair.

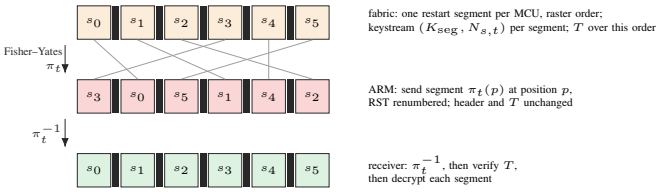


Fig. 5. E5 in hardware form: raster-order segments from the fabric, keyed order on the wire, raster order again before the tag check.

Protection levels. E0 only tags; E1 encrypts AC signs, E2 also DC signs (the default floor), E3 AC amplitudes, E4 DC amplitudes. Huffman symbols reveal where non-zero coefficients are, so E1–E4 leak outlines. E5 encrypts like E4 and sends the M macroblocks in a keyed order π_t (Fig. 5): Fisher–Yates with rejection sampling on the keystream of $(K_\pi, N(\sigma, t, 0))$ yields a uniform permutation, $\log_2 M! = 2{,}041$ bits at 320×240 ($M = 300$), new in every frame. Each macroblock is a restart segment, so the fabric needs no frame buffer; the ARM only reorders segments and the receiver restores raster order before checking T_t, so a moved segment fails the tag.

Lanes. One pipeline takes one sample per clock and 4:2:0 has 1.5 samples per pixel; a 64-bit input brings two pixels per clock. With L lanes the cost per pixel is therefore bounded by

$$c_L \geq \max\!\left(\tfrac{1.5}{L}, \tfrac{1}{2}\right) \text{ cycles/pixel}, \qquad \Theta_L = f_{\mathrm{clk}}/c_L, \tag{6}$$

i.e. at most 67, 133 and 200 Mpixel/s for $L = 1, 2, 4$ at 100 MHz. Restart segments make lanes independent (Fig. 2); the bytes do not depend on L.

Feedback and policy. The receiver checks a 64-frame replay window and returns authenticated heartbeats and alerts. Missing heartbeats for 3 s raise the top level and rekey (dead-man rule). A threshold policy escalates on tag failures, replays, loss patterns and forged feedback; a contextual bandit is the alternative.

IV. Implementation and Verification

A Python golden model passes all 1089 Ascon-AEAD128 known-answer tests; every RTL block has a cocotb testbench with random back-pressure that reports the first mismatch, and the encoders are byte-exact with the model in Icarus and Vivado XSim, for one, two and four lanes and all six levels. Encryption costs no cycles (E0–E4 differ by at most 2 cycles per frame); cycles per pixel follow $1.5 + 1.22/r$ for r MCU rows, 1.53 at 640×480. After place and route the encoder meets 100 MHz (WNS +0.630 ns): 65 Mpixel/s, 212 frames/s at VGA, 4,785 LUTs and about 206 mW with simulated switching activity (Table I). Two and four lanes reach 122 and 176 Mpixel/s in RTL simulation and close 100 MHz; E5 costs no BRAM, and with two lanes it draws 317 mW against 318 mW for E2. Both PYNQ-Z2 systems (Zynq PS, AXI DMA, encoder) close timing. Each Ascon core (Fig. 4) takes 860–970 LUTs (Table II).

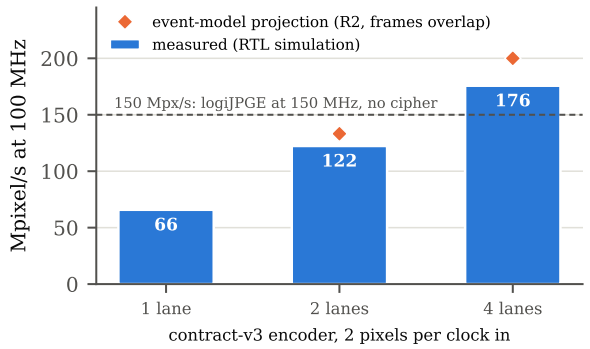


Fig. 6. Lane encoder throughput at 100 MHz, 640×480, cycle-accurate RTL simulation with every frame bit-exact (bars); diamonds: event-model projection with overlapping frames; dashed: a commercial encoder without cryptography [15].

TABLE I
RESOURCES ON THE XC7Z020 (53 200 LUT, 106 400 FF, 140 BRAM36, 220 DSP). SOURCE: VIVADO POST-ROUTE, OUT OF CONTEXT, 100 MHZ WITH I/O DELAYS OF 20 % OF THE PERIOD, WNS +0.630 NS.

Block	LUT	FF	BRAM36	DSP
Encoder (all)	4785 (9.0%)	3051 (2.9%)	25.5 (18.2%)	23 (10.4%)
colour conversion + line buffers	459	220	25	0
DCT and quantizer	806	69	0	22
zig-zag, Huffman, packing	717	323	0	0
keystream generator (Ascon)	949	851	0	0
tag unit (Ascon)	1358	1072	0	0
selective encryption	82	24	0	0
AXI-Lite registers	392	451	0	0

V. SECURITY EVALUATION

Setup. We score what a keyless observer can reconstruct: the stream decoded as is, and a *replacement attack* that substitutes fixed guesses for encrypted values. Leakage is the Pearson correlation of Sobel gradient magnitudes with the original (edge correlation, 0: no outline). Sources: a synthetic scene, 5 private clips, 2 Blender films and the 24 Kodak images. All levels decode without error in Pillow, OpenCV, scikit-image, FFmpeg (MJPEG).

Outline leakage. Under E1–E4 the eavesdropper sees the scene's outline (edge correlation 0.28–0.56, Table III, Fig. 7); encrypting amplitudes does not help, because positions and sizes stay public. E5 removes it (-0.007 to 0.014) for 30.9 % encrypted bits, against 29.5 % for E4, at 12–21 % more bytes for the restart markers.

Benchmark against published schemes. We re-implemented four format-compliant schemes [3], [4], [7], [9] with our keystream and scored all schemes under four attacks on 175 frames of 9 sources (Fig. 8): A1 decoding, A2 replacement, A3 non-zero counting (NZCA, [7]) and A4, which recovers the permutation and keystream bits from one known frame and applies them to the next frames. Every published scheme leaks 0.58–0.94 under its worst attack; fixed key material falls to A4, including Hirose et al. with one block per segment (0.90), although it costs as many bytes as E5. E5, keyed anew per frame, stays at or below 0.02 in both its model and hardware form.

TABLE II
COMPARISON WITH PUBLISHED FPGA DESIGNS. THEIR NUMBERS ARE AS REPORTED BY EACH SOURCE; OURS ARE POST-ROUTE (TABLE I) WITH THROUGHPUT FROM THE RTL CYCLE COUNT OF A 640×480 FRAME. MPX/S: MILLION PIXELS PER SECOND. LANES (V3): THROUGHPUT IN RTL SIMULATION AT 100 MHZ, AREA POST-ROUTE.

Design	Device	LUT	BRAM	DSP	MHz	Mpx/s	Crypto
Ours: encoder	Zynq-7020	4,785	25.5	23	100	65	sel. AEAD
Ours: one Ascon	Zynq-7020	860–970	0	0	100	–	AEAD core
Ours: 4 lanes (v3)	Zynq-7020	16,345	56.0	92	100	176	sel. AEAD
logiJPGE [15]	Zynq-7020	3,240	16	30	150	150	none
CAST JPEG-E-S [16]	Artix-7	3,069	2	16	166	–	none
Khairallah [17]	Artix-7	1,031	0	0	200	–	MAC
Kurian [13]	Artix-7	5,335	32	0	100	–	AEAD

TABLE III
OUTLINE LEAKAGE: EDGE CORRELATION BETWEEN THE ORIGINAL AND THE EAVESDROPPER'S BEST KEYLESS RECONSTRUCTION (DECODE OR REPLACEMENT ATTACK; 0 = NO OUTLINE), UP TO 60 FRAMES PER SOURCE; FRAMES WITHOUT EDGES ARE SKIPPED. THE FIRST ROW IS THE FRACTION OF PAYLOAD BITS ENCRYPTED. JIG.: SHARE OF NEIGHBOUR PAIRS A GREEDY JIGSAW SOLVER RESTORES UNDER E5 (8 FRAMES; SINTEL 3); ORACLE: EDGE CORRELATION IF EVERY E5 BLOCK WERE PUT BACK IN PLACE, I.E. THE OUTLINE E5 HIDES.

Source	E0	E1	E2	E3	E4	E5	Jig. (%)	Oracle
Enc. share	0.00	0.11	0.13	0.26	0.29	0.31		
Synthetic	1.00	0.38	0.36	0.38	0.38	0.00	2.3	0.37
Trailer A	0.99	0.47	0.30	0.32	0.28	0.01	1.4	0.46
Trailer B	0.99	0.56	0.34	0.33	0.32	0.01	1.3	0.41
Talk A	0.99	0.51	0.45	0.44	0.44	0.01	1.4	0.45
Talk B	0.99	0.49	0.40	0.42	0.40	0.01	1.7	0.47
Phone A	0.99	0.45	0.35	0.34	0.33	0.00	–	–
Sintel	0.99	0.35	0.29	0.31	0.28	-0.01	0.8	0.42
Tears of Steel	0.99	0.45	0.35	0.34	0.35	-0.01	1.3	0.41
Kodak	0.99	0.39	0.33	0.34	0.31	0.00	1.3	0.32

Stronger attackers. Greedy and minimum-spanning-tree jigsaw solvers [18], [19] place at most 1.7 and 2.5 % of E5 neighbours (chance), and one frame's exact permutation places 0.4 % of the next frame's tiles [8]. The linear program of Lin et al. recovers 59 % of E1's signs and 49 % of E2's (chance 50 %). A U-Net [20] trained on 3,417 frames and tested on unseen sources breaks E1 (edge 0.69, SSIM 0.79) but learns only brightness and colour under E5 (SSIM 0.34 against 0.36 for a frame-statistics oracle; edge 0.034, Table IV).

VI. PROTOCOL AND DEFENCE EVALUATION

Sender, network channel with attacker and receiver run as separate processes over UDP (320×240, 10 frames/s). Eleven attacks run 25 s apart: scan bit flips, mode downgrade, splicing, injection, replay, delay, dropping, junk flood, and dropping, forging or replaying feedback. Over 3 runs all were mitigated, on average 0.91 ± 0.02 s after they began, with 0/375 false alarms (Table V); integrity attacks are caught at the first frame, attacks by absence only by timeouts (dropped feedback, 3.5 s). Benign loss up to 6 %, delay and reordering raised no alarm (0/447 windows). The bandit encrypts 14 % fewer bits than the threshold rule but reacts later (1.34 against 0.46 s).

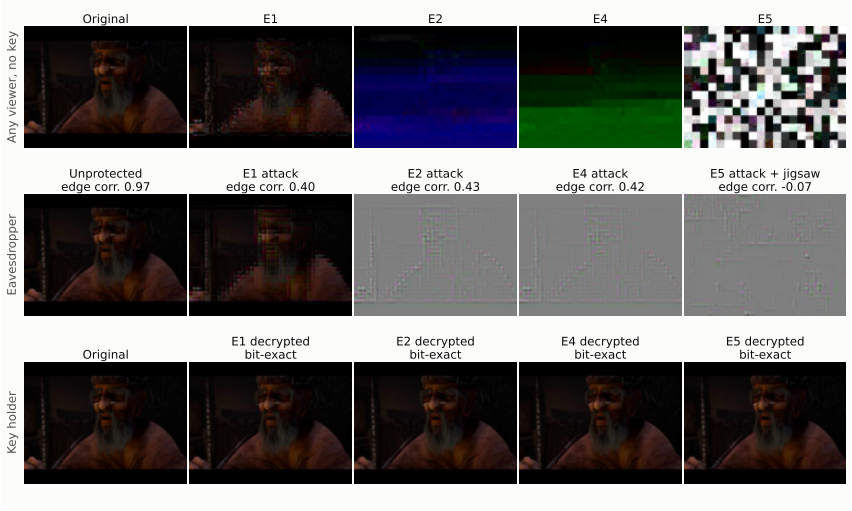


Fig. 7. One frame of the Sintel trailer (© Blender Foundation, CC BY 3.0). Top: any viewer without the key. Middle: the eavesdropper's best picture with its edge correlation (E5: after the jigsaw solver). Bottom: the key holder, bit-exact.

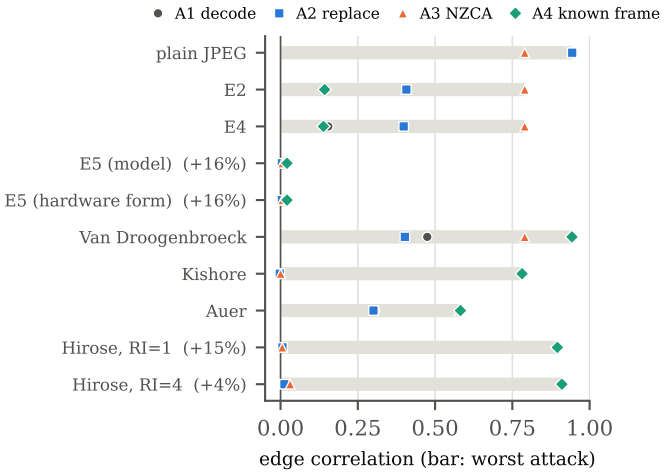


Fig. 8. SVE-Bench: edge correlation under each attack, pooled over 9 sources (byte cost over plain JPEG in brackets when above 2 %).

TABLE IV
U-NET RECONSTRUCTION, SSIM / EDGE CORR. ON UNSEEN TEST CLIPS. P1: TRAILER A→B, P2: TALK A→B, P3: SINTEL→TEARS OF STEEL, P4: FOUR SOURCES→THREE OTHERS. REPL.: REPLACEMENT ATTACK, P4 TEST SET. LAST ROW: THE MEDIAN TRAINING FRAME, SCORED WITHOUT LOOKING AT THE STREAM.

Mode	Repl.	P1	P2	P3	P4
E1	0.57 / 0.40	0.72 / 0.53	0.70 / 0.66	0.55 / 0.41	0.79 / 0.69
E2	0.21 / 0.39	0.42 / 0.33	0.27 / 0.34	0.14 / 0.31	0.37 / 0.43
E4	0.22 / 0.38	0.43 / 0.34	0.28 / 0.32	0.13 / 0.30	0.36 / 0.37
E5	0.18 / 0.01	0.34 / -0.02	0.26 / 0.02	0.22 / 0.00	0.34 / 0.03
No key, no stream	–	–	–	0.21 / 0.04	0.25 / -0.00

TABLE V
REACTION TO EACH ACTIVE ATTACK (FULL SUITE, THRESHOLD POLICY, 3 RUNS; MEAN ± S.D. IN SECONDS). "FIRST REJECTION" IS WHEN THE RECEIVER FIRST DISCARDS A FRAME; "MITIGATED" IS WHEN THE SENDER HAS RAISED PROTECTION OR REKEYED.

Attack	Mitigated	First rejection	Mitigated after
bit flips in the scan	3/3	0.09 ± 0.03	0.41 ± 0.02
mode downgrade in header	3/3	0.06 ± 0.02	0.44 ± 0.02
splice of two frames	3/3	0.13 ± 0.00	0.46 ± 0.01
forged frame injection	3/3	0.14 ± 0.06	0.44 ± 0.03
frame replay	3/3	0.07 ± 0.05	0.46 ± 0.03
frame delay	3/3	4.13 ± 0.04	1.46 ± 0.03
frame dropping	3/3	–	1.46 ± 0.03
junk flood	3/3	–	0.49 ± 0.04
feedback dropping	3/3	–	3.47 ± 0.05
forged feedback	3/3	–	0.46 ± 0.04
feedback replay	3/3	–	0.44 ± 0.01

VII. LIMITATIONS AND CONCLUSION

On the PYNQ-Z2 the two-lane E5 bitstream is bit-exact with the golden model in E0–E5 at 64×48, 320×240 and 640×480 (84 frames). A 640×480 frame takes 92 ms with our Python driver (10.8 frames/s; 45 at 320×240), the same in every level within 0.09 ms, so encryption costs no time on silicon either; this is the ARM's pixel copy, not the fabric (2.5 ms in RTL). Board power and the ARM baseline are pending [TODO: GUIDE.md A3, A4], so our power figures come from simulated switching activity. No level hides frame sizes: ten sizes identify one of 7 videos 98 % of the time under E2 (99 % under E5), and only constant-size frames prevent it, at up to +179 % bytes on edited clips. Our attackers are greedy or trained on few clips.

Encrypting coefficient fields with Ascon inside a JPEG encoder costs no cycles, keeps every level playable and, with a per-frame tag, turns MJPEG into an authenticated stream. Published format-compliant schemes fall to non-zero counting or one known frame; a per-frame keyed shuffle of restart segments, built in hardware without a frame buffer, does not.

ACKNOWLEDGMENT

We thank Dr. Chetana G for her guidance. [TODO: AI-assistance disclosure as required by the conference.]

REFERENCES

[1] M. Baugher, D. McGrew, M. Naslund, E. Carrara, and K. Norrman, "The secure real-time transport protocol (SRTP)," IETF, RFC 3711, Mar. 2004.

[2] A. Massoudi, F. Lefebvre, C. De Vleeschouwer, B. Macq, and J.-J. Quisquater, "Overview on selective encryption of image and video: Challenges and perspectives," *EURASIP Journal on Information Security*, vol. 2008, p. 179290, 2008.

[3] M. Van Droogenbroeck and R. Benedett, "Techniques for a selective encryption of uncompressed and compressed images," in *Proc. Advanced Concepts for Intelligent Vision Systems (ACIVS)*, Ghent, Belgium, Sep. 2002, pp. 90–97.

[4] S. Auer, A. Bliem, D. Engel, and A. Unterweger, "Bitstream-based JPEG encryption in real-time," *International Journal of Digital Crime and Forensics*, vol. 5, no. 3, pp. 1–14, 2013.

[5] R. Lin, S. Liu, J. Jiang, S. Li, C. Li, and C.-C. J. Kuo, "Recovering sign bits of DCT coefficients in digital images as an optimization problem," *Journal of Visual Communication and Image Representation*, p. 104045, 2024.

[6] M. S. Turan, K. A. McKay, D. Chang, J. Kang, and J. Kelsey, "Ascon-based lightweight cryptography standards for constrained devices: Authenticated encryption, hash, and extendable output functions," National Institute of Standards and Technology, NIST Special Publication 800-232, Aug. 2025.

[7] M. Hirose, S. Imaizumi, and H. Kiya, "On the security of bitstream-level JPEG encryption with restart markers," in *Proc. Asia Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC)*, 2024, arXiv:2410.06522.

[8] S. Li, Y. Guo, J. Huang, and R. Gao, "Fast attack algorithm for JPEG image encryption with block position shuffle," *Journal of Shanghai Jiaotong University (Science)*, vol. 30, no. 5, pp. 976–987, 2025.

[9] B. Kishore, B. K. Shreyamsha Kumar, and C. R. Patil, "FPGA based simple and fast JPEG encryptor," *Journal of Real-Time Image Processing*, vol. 10, no. 3, pp. 551–559, 2015.

[10] A. Ç. Bağbaba and B. Örs, "Hardware implementation of novel image compression-encryption system on a FPGA," in *Proc. 9th International Conference on Electrical and Electronics Engineering (ELECO)*, Bursa, Turkey, 2015, pp. 1159–1163.

[11] H. Groß, E. Wenger, C. Dobraunig, and C. Ehrenhöfer, "Ascon hardware implementations and side-channel evaluation," *Microprocessors and Microsystems*, vol. 52, pp. 470–479, 2017.

[12] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer, "Ascon v1.2: Lightweight authenticated encryption and hashing," *Journal of Cryptology*, vol. 34, no. 3, p. 33, 2021.

[13] M. Gladis Kurian and Y. Chen, "Ascon on FPGA: Post-quantum safe authenticated encryption with replay protection for IoT," *Electronics*, vol. 14, no. 13, p. 2668, 2025.

[14] R. Schuster, V. Shmatikov, and E. Tromer, "Beauty and the burst: Remote identification of encrypted video streams," in *Proc. 26th USENIX Security Symposium*, 2017, pp. 1357–1374.

[15] Xylon d.o.o., "logiJPGE motion JPEG encoder data sheet, v1.0," https://www.logicbricks.com/Documentation/Datasheets/IP/logiJPGE_hds.pdf, 2020.

[16] CAST, Inc., "JPEG-E-S baseline JPEG encoder, product brief," https://www.cast-inc.com/pdfs/cast_jpeg-e-s-brief-amd.pdf, accessed 14 Oct. 2026.

[17] M. Khairallah and S. Yadhunathan, "FPGA implementations of message authentication codes based on Ascon-p," in *NIST Lightweight Cryptography Workshop*, 2023.

[18] D. Pomeranz, M. Shemesh, and O. Ben-Shahar, "A fully automated greedy square jigsaw puzzle solver," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2011, pp. 9–16.

[19] A. C. Gallagher, "Jigsaw puzzles with pieces of unknown orientation," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012, pp. 382–389.

[20] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, ser. LNCS, vol. 9351, 2015, pp. 234–241.